

Общее описание проекта

StudentApi — это backend-сервис на [ASP.NET](#) Core Web API для управления процессом подачи заявок на практику студентами. Он выполняет функции:

- Регистрации и аутентификации студентов.
- Создания заявок на практику.
- Заполнения анкет (расширенная информация о студенте).
- Интеграции с внешней системой ManagerApi (условный сервис руководителей практики).
- Хранения данных в PostgreSQL через Entity Framework Core.

1. Контроллеры (API endpoints)

ApplicationController

Управляет заявками студентов.

csharp

```
[ApiController]
```

```
[Route("api/v1/{controller}")]
```

```
public class ApplicationController : ControllerBase
```

Эндпоинты:

- POST /api/v1/application — создание новой заявки. Проверяет модель, вызывает сервис, возвращает 201 Created.
- POST /api/v1/application/{id}/submit — отправка заявки менеджеру. Изменяет статус с «Заявка» на «НаРассмотренииМенеджером».
- GET /api/v1/application/student/{studentId} — все заявки конкретного студента.
- GET /api/v1/application/{id} — детали одной заявки.

Комментарий: Хорошо использован CreatedAtAction для возврата ссылки на созданный ресурс.

AuthController

Регистрация и вход.

csharp

```
[HttpPost("register")]
```

```
public async Task<IActionResult> Register([FromBody] StudentRegisterDto dto)
```

- При регистрации пароль хэшируется (SHA256 в AuthService).
- При логине проверяется хэш.
- Возвращается StudentResponseDto без пароля.

Комментарий: SHA256 — не лучший выбор для паролей в production (лучше BCrypt или PBKDF2). Но для учебного проекта пойдёт.

ReferenceController

Получает справочные данные из ManagerApi:

- Типы практик
- Специализации
- Запланированные практики (с фильтрацией по типу и специализации)

Комментарий: Здесь используется ManagerApiClient для HTTP-запросов к внешнему сервису. Есть кэширование в ManagerDataService.

QuestionnaireController

CRUD для анкет студентов (расширенная информация: родственники, образование, места работы и т.д.).

Особенности:

- POST /api/v1/questionnaire — создание анкеты. Проверяет, что заявка существует и анкета ещё не создана.
- GET /api/v1/questionnaire/application/{applicationId} — анкета по ID заявки (может вернуть null).
- PUT /api/v1/questionnaire/{id} — обновление. При обновлении **удаляются старые связанные сущности** (родственники, навыки и т.п.) и создаются заново. Это упрощает логику, но может быть неэффективно при большом количестве данных.
- DELETE /api/v1/questionnaire/{id} — удаление анкеты.

Комментарий: UpdateQuestionnaireAsync вызывает `_context.Relatives.RemoveRange(...)`. Это рабочий, но не самый оптимальный подход. Альтернатива — сравнивать старые и новые списки и обновлять только изменённые элементы.

2. DTO (Data Transfer Objects)

DTO разделены на:

- **Входные** (CreateApplicationDto, StudentRegisterDto, CreateQuestionnaireDto).
- **Выходные** (ApplicationResponseDto, StudentResponseDto, QuestionnaireResponseDto).

Плюсы:

- Скрывают внутренние модели (например, скрывают PasswordHash).
- Позволяют валидировать данные через атрибуты ([Required], [EmailAddress] и т.д.).

Комментарий: В CreateApplicationDto поля IdScheduledPractice, IdPracticeType, IdSpecialization сделаны обязательными ([Required]), хотя в модели StudentApplication они int?. Это нормально для бизнес-логики создания заявки.

3. Модели базы данных (Entities)

Файлы в StudentApi/Models/:

- Student — основная информация о студенте (хэш пароля, ФИО, контакты).
- StudentApplication — заявка на практику. Содержит статус (enum StudentApplicationStatus), желаемые даты, внешние ключи на Student и на справочники из ManagerApi.
- Questionnaire — основная анкета. Связана с StudentApplication (один к одному или null).
- PsychologicalQuestions, Relative, Education, PlacePractice, PlaceWork, Skill, StudentProject, PracticePriority — связанные сущности анкеты.

Комментарий: В StudentApplication есть дублирующиеся поля IdStudent (два раза). Это опечатка в файле. Нужно оставить одно.

Настройки отношений в AppDbContext.OnModelCreating:

- StudentApplication — cascade delete при удалении студента.
- Questionnaire — SetNull при удалении StudentApplication (анкета остаётся, но теряет связь с заявкой).
- Все дочерние сущности анкеты — cascade delete.

4. Сервисы (Business Logic)

AuthService

- HashPassword — SHA256.
- RegisterAsync — проверяет уникальность логина, создаёт студента.
- LoginAsync — проверяет логин и пароль.

ApplicationService

- CreateApplicationAsync — проверяет существование студента и корректность дат.
- SubmitApplicationAsync — отправляет заявку в ManagerApiClient. При успехе меняет статус.
- GetApplicationAsync — получает заявку и при необходимости запрашивает актуальный статус у менеджера.
- UpdateApplicationStatus — маппинг статусов из строки (от ManagerApi) в enum.

Комментарий: В SubmitApplicationAsync статус меняется только если заявка в состоянии Заявка. Это защита от повторной отправки.

ManagerApiClient

HTTP-клиент для взаимодействия с ManagerApi:

- GetPracticeTypesAsync, GetSpecializationsAsync, GetScheduledPracticesAsync — получение справочников.
- SendApplicationToManagerAsync — POST-запрос на .../ReceiveApplication.
- GetApplicationStatusAsync — GET-запрос на .../GetApplicationStatus/{id}.
- SendQuestionnaireToManagerAsync — отправка анкеты.

Комментарий: URL-адреса захардкожены в методах. Лучше вынести их в конфигурацию или в константы класса.

ManagerDataService

Прослойка с кэшированием справочников (PracticeType, Specialization). Кэш живёт 5 минут. Это уменьшает количество запросов к ManagerApi.

QuestionnaireService

- CreateQuestionnaireAsync — маппинг DTO → Entity через MapToEntity. Проверяет, что заявка существует и анкета ещё не создана.
- GetQuestionnaireByIdAsync — загружает все связанные сущности через Include и ThenInclude.
- UpdateQuestionnaireAsync — удаляет старые связанные данные, затем обновляет основные поля и добавляет новые коллекции. **Важный момент:** после удаления не вызывает SaveChanges до добавления новых, поэтому всё происходит в одной транзакции.
- DeleteQuestionnaireAsync — удаляет анкету.

Комментарий: В методе UpdateQuestionnaireAsync удаление и вставка новых записей — простой, но неоптимальный паттерн. При большом количестве данных лучше реализовать частичное обновление.

5. Программа и настройки (Program.cs)

Основные блоки:

```
csharp
```

```
// Подключение БД
```

```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
```

```
builder.Services.AddDbContext<AppDbContext>(options => options.UseNpgsql(connectionString));
```

```
// Регистрация сервисов
```

```
builder.Services.AddScoped<AuthService>();
```

```
builder.Services.AddScoped<ApplicationService>();
```

```
// ...
```

```
// HTTP-клиент для ManagerApi
```

```
builder.Services.AddHttpClient<ManagerApiClient>(client =>
```

```
{
```

```
    client.BaseAddress = new Uri(managerApiBaseUrl);
```

```
});
```

```
// CORS для Vue.js фронтенда
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowVueApp", policy =>
    {
        policy.WithOrigins("http://localhost:5173", "http://localhost:5174")
            .AllowAnyHeader().AllowAnyMethod().AllowCredentials();
    });
});

// Автоматическая миграция БД при старте
using (var scope = app.Services.CreateScope())
{
    var dbContext = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    await dbContext.Database.MigrateAsync();
}
```

Комментарий: Используется `MigrateAsync()`, а не `EnsureCreatedAsync()` — правильно для production, так как применяет миграции.

6. Файл `appsettings.json`

```
json
{
    "ConnectionStrings": {
        "DefaultConnection":
        "Host=82.22.38.155;Port=5432;Database=Student_DB;Username=student;Password=12345"
    },
    "Services": {
        "ManagerApi": "http://localhost:5001"
    }
}
```

Комментарий: Не храните реальные пароли в коде! В production используйте User Secrets, переменные окружения или Azure Key Vault.

7. Замечания и потенциальные проблемы

1. **Дублирование поля IdStudent в StudentApplication** — удалите лишнюю строку.
2. **Хэширование паролей** — SHA256 недостаточно стоек к атакам. Используйте BCrypt.Net или Microsoft.AspNetCore.Identity.
3. **Обновление анкеты** — полное удаление и вставка может привести к потере производительности и сбросу Identity-ключей. Рассмотрите «умное» обновление.
4. **Обработка ошибок** — в контроллерах много try-catch, но в некоторых местах (например, QuestionnaireService.CreateQuestionnaireAsync) ошибка просто пробрасывается (throw;). Лучше возвращать null или кастомный DTO с флагом ошибки.
5. **Логирование** — настроено хорошо: есть ILogger в сервисах и контроллерах.
6. **URL ManagerApi** — читается из конфигурации, но внутри методов ManagerApiClient строки типа "api/v1/PracticeTypes/GetPracticeTypes" захардкожены. Вынесите их в конфигурацию или статические поля.
7. **Опечатки в статусах** — в ApplicationService.UpdateApplicationStatus написано "Approved" => StudentApplicationStatus.Primant (должно быть Принят). Проверьте все маппинги.

Итог

Код написан грамотно, с соблюдением архитектурных принципов (контроллеры → сервисы → репозиторий через EF Core). Проект готов к использованию в учебных или небольших реальных задачах после исправления указанных недочётов.

Сильные стороны:

- Чёткое разделение ответственности.
- Использование DTO для безопасности.
- Интеграция с внешним API через отдельный клиент.
- Логирование ошибок.
- CORS для фронтенда.

Что можно улучшить:

- Перейти на асинхронное хэширование паролей.
- Реализовать частичное обновление анкеты.
- Убрать дублирование кода маппинга (использовать AutoMapper).

Ниже представлен **построчный и поблочный комментарий** к ключевым участкам кода, который поможет вам уверенно отвечать на вопросы на защите. Комментарии написаны так, чтобы показать **понимание архитектурных решений, безопасности и логики работы**.

1. ApplicationController.cs (контроллер заявок)

csharp

```
[ApiController] // Включает автоматическую валидацию модели, привязку данных
[Route("api/v1/{controller}")] // Шаблон маршрута: /api/v1/application

public class ApplicationController : ControllerBase
{
    private readonly ApplicationService _applicationService; // Внедрение зависимости

    // POST /api/v1/application - создание новой заявки
    [HttpPost]
    public async Task<ActionResult> CreateApplication([FromBody] CreateApplicationDto dto)
    {
        if (!ModelState.IsValid) // Проверка атрибутов [Required] и т.д.
            return BadRequest(ModelState); // Возвращаем ошибки валидации

        var result = await _applicationService.CreateApplicationAsync(dto);

        if (result == null)
            return BadRequest(new { message = "Не удалось создать заявку" });

        // 201 Created + заголовок Location с URL новой заявки
        return CreatedAtAction(nameof(GetApplication), new { id = result.IdStudentApplication }, result);
    }

    // POST /api/v1/application/{id}/submit - отправка менеджеру
    [HttpPost("{id}/submit")]
    public async Task<ActionResult> SubmitApplication(int id)
    {
        var result = await _applicationService.SubmitApplicationAsync(id);
```



```

        if (result == null)
            return NotFound(new { message = "Заявка не найдена" });
        return Ok(result);
    }
}

```

Что важно для защиты:

- CreatedAtAction — стандарт REST, возвращает URL нового ресурса
- ModelState.IsValid — проверка DTO до передачи в сервис
- Асинхронность везде (async/await) — не блокируем потоки

2. AuthController.cs (аутентификация)

```

csharp

[ApiController]
[Route("api/v1/[controller]")] // [controller] = Auth -> /api/v1/auth
public class AuthController : ControllerBase
{
    // POST /api/v1/auth/register
    [HttpPost("register")]
    public async Task<IActionResult> Register([FromBody] StudentRegisterDto dto)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);

        var result = await _authService.RegisterAsync(dto);

        if (result == null)
            return Conflict(new { message = "Пользователь с таким логином уже существует" });

        return CreatedAtAction(nameof(Register), new { id = result.IdStudent }, result);
    }

    // POST /api/v1/auth/login

```

```

[HttpPost("login")]
public async Task<ActionResult> Login([FromBody] StudentLoginDto dto)
{
    var result = await _authService.LoginAsync(dto);

    if (result == null)
        return Unauthorized(new { message = "Неверный логин или пароль" });

    return Ok(result); // Возвращаем данные студента (без пароля)
}
}

```

Ключевые моменты:

- Conflict (409) — логин уже существует, правильный HTTP-статус
- Unauthorized (401) — неверные учётные данные
- Пароль не возвращается никогда — только StudentResponseDto без поля PasswordHash

3. ReferenceController.cs (справочники)

csharp

```

[HttpGet("practicetypes")]
public async Task<ActionResult> GetPracticeTypes()
{
    try
    {
        var types = await _managerDataSource.GetPracticeTypesAsync();
        return Ok(types);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error getting practice types");
        return StatusCode(500, new { message = "Ошибка при получении типов практик" });
    }
}

```

Что демонстрируем:

- Логирование ошибок с уровнем Error
 - StatusCode(500) — внутренняя ошибка сервера
 - Прокси-паттерн: контроллер делегирует работу ManagerDataService
-

4. QuestionnaireController.cs (анкеты)

csharp

```
[HttpGet("application/{applicationId}")]
```

```
public async Task<IActionResult> GetQuestionnaireByApplication(int applicationId)
```

```
{
```

```
    try
```

```
    {
```

```
        var result = await _questionnaireService.GetQuestionnaireByApplicationIdAsync(applicationId);
```

```
        if (result == null)
```

```
            return Ok(null); // Анкета ещё не заполнена — это не ошибка
```

```
        return Ok(result);
```

```
    }
```

```
    catch (Exception ex)
```

```
    {
```

```
        _logger.LogError(ex, $"Error getting questionnaire for application {applicationId}");
```

```
        return StatusCode(500, new { message = "Внутренняя ошибка сервера" });
```

```
    }
```

```
}
```

```
[HttpPut("{id}")]
```

```
public async Task<IActionResult> UpdateQuestionnaire(int id, [FromBody] CreateQuestionnaireDto dto)
```

```
{
```

```
    // Сначала удаляем старые связанные данные, потом создаём новые
```

```
    // Это упрощает обновление сложной иерархии
```

```
}
```

Важно для защиты:

- Ok(null) — осознанное решение: отсутствие анкеты не ошибка, а состояние
 - При обновлении используется стратегия "удали всё и создай заново" — проще, но не оптимально для больших данных
-

5. DTO и валидация

csharp

```
public class StudentRegisterDto
{
    [Required] // Без этого поля запрос не пройдет валидацию
    [MaxLength(255)]
    public string Surname { get; set; } = string.Empty; // = "" - защита от null

    [Required]
    [MinLength(6)] // Минимальная длина пароля
    public string Password { get; set; } = string.Empty; // Обычный пароль, не хэш!

    [Required]
    [EmailAddress] // Встроенная валидация формата email
    public string Email { get; set; } = string.Empty;

    [Required]
    [Phone] // Валидация номера телефона
    public string PhoneNumber { get; set; } = string.Empty;
}
```

Зачем это нужно:

- Входные DTO содержат **сырой пароль**, выходные — никогда
 - Атрибуты валидации срабатывают до вызова контроллера
 - string.Empty вместо null — избегаем NullReferenceException
-

6. Модели БД (Entity)

csharp

```
[Table("Student")]
```

```

public class Student
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)] // БД сама генерирует ID
    public int IdStudent { get; set; }

    [Required]
    [MaxLength(255)]
    public string Login { get; set; } = string.Empty;

    [Required]
    public string PasswordHash { get; set; } = string.Empty; // Только хэш, не пароль!

    [Column(TypeName = "timestamp without time zone")] // Для PostgreSQL
    public DateTime Birthdate { get; set; }
}

```

Обратите внимание:

- PasswordHash — никогда не храним пароли в открытом виде
- Явное указание типа для PostgreSQL (без часового пояса)
- Навигационные свойства (virtual) для Entity Framework

csharp

```

public enum StudentApplicationStatus
{
    Заявка, // Студент создал, но не отправил
    НаРассмотренииМенеджером, // Отправлена менеджеру
    Тестирование, // Менеджер назначил тест
    СобеседованиеСМенеджером,
    НаРассмотренииРуководителем,
    СобеседованиеСРуководителем,
    ОформлениеДокументов,
    Принят,
    Отказано
}

```

```
}
```

Статусы на русском — для удобства фронтенда, в БД хранятся как `int`.

7. Настройка отношений в `AppDbContext`

csharp

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Уникальные индексы
    modelBuilder.Entity<Student>()
        .HasIndex(s => s.Login)
        .IsUnique(); // Нельзя зарегистрироваться с одинаковым логином

    // Каскадное удаление
    modelBuilder.Entity<StudentApplication>()
        .HasOne(sa => sa.Student)
        .WithMany()
        .HasForeignKey(sa => sa.IdStudent)
        .OnDelete(DeleteBehavior.Cascade); // Удалили студента → удалили заявки

    // При удалении заявки анкета остаётся, но теряет связь
    modelBuilder.Entity<Questionnaire>()
        .HasOne(q => q.StudentApplication)
        .WithOne()
        .HasForeignKey<Questionnaire>(q => q.IdStudentApplication)
        .OnDelete(DeleteBehavior.SetNull); // SetNull, не Cascade!
}
```

Почему SetNull:

- Анкета — ценный документ, её нельзя удалять вместе с заявкой
- Если студент отозвал заявку, анкета остаётся в системе

8. `AuthService` — хэширование паролей

csharp

```

private string HashPassword(string password)
{
    using var sha256 = SHA256.Create(); // Алгоритм хэширования
    var hashedBytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
    return Convert.ToBase64String(hashedBytes);
}

public async Task<StudentResponseDto> RegisterAsync(StudentRegisterDto dto)
{
    // Проверка уникальности логина
    var existingStudent = await _context.Students
        .FirstOrDefaultAsync(s => s.Login == dto.Login);
    if (existingStudent != null)
        return null; // Конфликт

    var student = new Student
    {
        // ...
        PasswordHash = HashPassword(dto.Password), // Храним только хэш!
    };
}

```

Почему SHA256 — не идеал, но допустимо:

- Соль не используется (можно добавить)
- Для учебного/внутреннего проекта достаточно
- В production лучше BCrypt или [ASP.NET Core Identity](#)

9. ApplicationService — бизнес-логика

```

csharp

public async Task<ApplicationResponseDto> SubmitApplicationAsync(int applicationId)
{
    var application = await _context.StudentApplications
        .Include(a => a.Student) // Жадная загрузка, чтобы не делать второй запрос

```

```

        .FirstOrDefaultAsync(a => a.IdStudentApplication == applicationId);

    if (application == null || application.Student == null)
        return null;

    if (application.StudentApplicationStatus == StudentApplicationStatus.Заявка) // Только черновик
    {
        var sent = await _managerApiClient.SendApplicationToManagerAsync(application,
            application.Student);

        if (sent)
        {
            application.StudentApplicationStatus = StudentApplicationStatus.НаРассмотренииМенеджером;
            application.UpdatedAt = DateTime.Now;
            await _context.SaveChangesAsync(); // Сохраняем изменения
        }
    }

    return MapToResponseDto(application, application.Student);
}

```

Проверка статуса — нельзя отправить уже отправленную заявку повторно.

10. ManagerApiClient — HTTP-клиент

```

csharp

public async Task<bool> SendApplicationToManagerAsync(StudentApplication application, Student
student)
{
    try
    {
        var applicationDto = new
        {
            studentId = student.IdStudent,
            studentName = $"{student.Surname} {student.Name} {student.Patronymic}".Trim(),

```



```

        applicationId = application.IdStudentApplication,
        submittedAt = DateTime.UtcNow // UTC — единый стандарт времени
    };

    var json = JsonSerializer.Serialize(applicationDto, _jsonOptions);
    var content = new StringContent(json, Encoding.UTF8, "application/json");

    var response = await _httpClient.PostAsync("api/v1/ManagerApplication/ReceiveApplication",
content);

    return response.IsSuccessStatusCode; // true для 200-299
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error sending application to Manager API");
    return false; // Не падаем, а логируем и возвращаем false
}
}

```

Ключевые решения:

- Анонимный тип вместо DTO — для простоты
- UtcNow — все серверы должны использовать UTC
- Исключения не выбрасываются наружу, а логируются

11. QuestionnaireService — сложное обновление

csharp

```

public async Task<QuestionnaireResponseDto?> UpdateQuestionnaireAsync(int id,
CreateQuestionnaireDto dto)
{
    var questionnaire = await _context.Questionnaires
        .Include(q => q.Relatives) // Загружаем все связанные коллекции
        .Include(q => q.Educations)
        // ...
        .FirstOrDefaultAsync(q => q.IdQuestionnaire == id);
}

```

```

// Удаляем старые связанные данные
_context.Relatives.RemoveRange(questionnaire.Relatives);
_context.Educations.RemoveRange(questionnaire.Educations);
// ...

// Обновляем основные поля
UpdateEntityFromDto(questionnaire, dto);

await _context.SaveChangesAsync(); // Всё в одной транзакции
}

```

Почему удаляем всё:

- Упрощает код — не нужно сравнивать старые и новые списки
- Для анкет с небольшим объёмом данных это приемлемо
- Минус: теряются старые ID, но это не важно

12. Program.cs — настройка приложения

csharp

```

// 1. Регистрация сервисов в DI-контейнере
builder.Services.AddScoped<AuthService>();    // Один объект на HTTP-запрос
builder.Services.AddScoped<ApplicationService>();
builder.Services.AddHttpClient<ManagerApiClient>(); // Специальная регистрация для HTTP-клиента

// 2. Настройка CORS для Vue.js
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowVueApp", policy =>
    {
        policy.WithOrigins("http://localhost:5173", "http://localhost:5174")
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials(); // Разрешаем куки
    });
});

```

```

});

});

// 3. Автоматическая миграция БД при старте
using (var scope = app.Services.CreateScope())
{
    var dbContext = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    await dbContext.Database.MigrateAsync(); // Применяет все ожидающие миграции
}

// 4. Конвейер middleware
app.UseCors("AllowVueApp"); // Сначала CORS
app.UseAuthorization();    // Потом авторизация
app.MapControllers();      // Затем маршрутизация

```

Порядок middleware важен: CORS → Auth → Endpoints.

13. appsettings.json — конфигурация

```

json
{
    "ConnectionStrings": {
        "DefaultConnection":
        "Host=82.22.38.155;Port=5432;Database=Student_DB;Username=student;Password=12345"
    },
    "Services": {
        "ManagerApi": "http://localhost:5001" // Адрес внешнего сервиса
    }
}

```

Для защиты проекта:

- В реальном проекте пароль БД не хранят в коде
- Используют User Secrets, переменные окружения или Azure Key Vault
- localhost:5001 — при деплое заменяется на реальный URL